



SETUP GUIDE

GitHub Actions for ML CI/CD Pipeline

SECTION 1: OVERVIEW

What we build:

- A GitHub repository with ML code, tests, and a CI/CD workflow
- A GitHub Actions workflow that runs on push/PR with 3 pipeline stages
- Unit tests for preprocessing and prediction code
- Data validation script (schema + quality checks)
- Model validation script (accuracy threshold check)
- Dockerfile for containerised model serving

Developer pushes code

SECTION 2: PREREQUISITES

Required:

- GitHub account (free tier works)
- Git installed locally
- Python 3.9+ installed locally (for local testing)
- Docker Desktop installed (for local container testing)
- Text editor (VS Code recommended)

GitHub Actions free tier includes:

- 2,000 minutes/month for private repos
- Unlimited for public repos
- Linux runners with 2 CPUs, 7GB RAM

No additional services needed — everything runs on GitHub's infrastructure.

SECTION 3: REPOSITORY STRUCTURE

Create this structure:



```
ml-cicd-demo/  
└─ .github/
```

SECTION 4: FILE CONTENTS

FILE: .github/workflows/ml-pipeline.yml

name: ML CI/CD Pipeline

on:

push:

branches: [main]

pull_request:

branches: [main]

jobs:

Stage 1: Test

test:

runs-on: ubuntu-latest

steps:

- name: Checkout code

uses: actions/checkout@v4

- name: Set up Python

uses: actions/setup-python@v5

with:

python-version: "3.11"

cache: "pip"

- name: Install dependencies

run: pip install -r requirements.txt

- name: Run unit tests

run: pytest tests/ -v --tb=short

- name: Validate data schema

run: python scripts/validate_data.py

Stage 2: Validate Model

validate:

needs: test

runs-on: ubuntu-latest



steps:

- name: Checkout code
uses: actions/checkout@v4
- name: Set up Python
uses: actions/setup-python@v5
with:
python-version: "3.11"
cache: "pip"
- name: Install dependencies
run: pip install -r requirements.txt
- name: Train and validate model
run: python scripts/validate_model.py

Stage 3: Build Container

build:

needs: validate

runs-on: ubuntu-latest

steps:

- name: Checkout code
uses: actions/checkout@v4
- name: Build Docker image
run: docker build -t ml-model:\${{ github.sha }} .
- name: Run container health check
run: |
docker run -d --name test-container -p 8080:8080 ml-model:\${{ github.sha }}
sleep 5
curl -f http://localhost:8080/health || exit 1
docker stop test-container

FILE: requirements.txt

scikit-learn==1.4.0

pandas==2.2.0

numpy==1.26.4

pytest==8.0.0

flask==3.0.2

FILE: src/__init__.py

FILE: src/preprocess.py

```
"""Data preprocessing functions for the ML pipeline."""
```

```
import pandas as pd
```

```
import numpy as np
```

```
def load_data(filepath: str) -> pd.DataFrame:
```

```
    """Load CSV data with basic validation."""
```

```
    df = pd.read_csv(filepath)
```

```
    if df.empty:
```

```
        raise ValueError(f"Empty dataset: {filepath}")
```

```
    return df
```

```
def handle_missing_values(df: pd.DataFrame, strategy: str = "median") -> pd.DataFrame:
```

```
    """Handle missing values in numerical columns."""
```

```
    numeric_cols = df.select_dtypes(include=[np.number]).columns
```

```
    if strategy == "median":
```

```
        df[numeric_cols] = df[numeric_cols].fillna(df[numeric_cols].median())
```

```
    elif strategy == "mean":
```

```
        df[numeric_cols] = df[numeric_cols].fillna(df[numeric_cols].mean())
```

```
    elif strategy == "drop":
```

```
        df = df.dropna(subset=numeric_cols)
```

```
    else:
```

```
        raise ValueError(f"Unknown strategy: {strategy}")
```

```
    return df
```

```
def normalize_features(df: pd.DataFrame, columns: list) -> pd.DataFrame:
```

```
    """Min-max normalize specified columns to [0, 1] range."""
```

```
    for col in columns:
```

```
        if col not in df.columns:
```

```
            raise KeyError(f"Column '{col}' not found in DataFrame")
```

```
        min_val = df[col].min()
```

```
        max_val = df[col].max()
```

```
        if max_val - min_val == 0:
```

```
            df[col] = 0.0
```

```
        else:
```

```
            df[col] = (df[col] - min_val) / (max_val - min_val)
```

```
    return df
```

```
def validate_schema(df: pd.DataFrame, expected_columns: list) -> bool:
    """Validate that DataFrame has expected columns."""
    missing = set(expected_columns) - set(df.columns)
    if missing:
        raise ValueError(f"Missing columns: {missing}")
    return True
```

FILE: src/train.py

```
"""Model training script."""

import pickle
import pandas as pd
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

from src.preprocess import load_data, handle_missing_values

def train_model(data_path: str, model_path: str = "model.pkl") -> dict:
    """Train a Random Forest classifier and save it."""
    # Load and preprocess
    df = load_data(data_path)
    df = handle_missing_values(df)

    # Split features and target (last column is target)
    X = df.iloc[:, :-1]
    y = df.iloc[:, -1]

    # Train/test split
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42
    )

    # Train model
    model = RandomForestClassifier(n_estimators=100, random_state=42)
    model.fit(X_train, y_train)

    # Evaluate
    y_pred = model.predict(X_test)
    accuracy = accuracy_score(y_test, y_pred)

    # Save model
```

```
with open(model_path, "wb") as f:  
    pickle.dump(model, f)
```

```
return {  
    "accuracy": accuracy,  
    "n_features": X.shape[1],  
    "n_samples": len(df),  
    "model_path": model_path,  
}
```

```
if __name__ == "__main__":  
    metrics = train_model("data/sample_data.csv")  
    print(f"Training complete. Accuracy: {metrics['accuracy']:.4f}")
```

FILE: src/predict.py

```
"""Inference/prediction server using Flask."""
```

```
import pickle  
import numpy as np  
from flask import Flask, request, jsonify
```

```
app = Flask(__name__)
```

```
# Load model at startup  
MODEL_PATH = "model.pkl"  
model = None
```

```
def load_model():  
    """Load the trained model from disk."""  
    global model  
    with open(MODEL_PATH, "rb") as f:  
        model = pickle.load(f)
```

```
@app.route("/health")  
def health():  
    """Health check endpoint."""  
    return jsonify({"status": "healthy", "model_loaded": model is not None})
```

```
@app.route("/predict", methods=["POST"])  
def predict():  
    """Make a prediction."""  
    if model is None:
```

```
return jsonify({"error": "Model not loaded"}), 503

data = request.get_json()
if "features" not in data:
    return jsonify({"error": "Missing 'features' field"}), 400

features = np.array(data["features"]).reshape(1, -1)
prediction = model.predict(features)[0]
confidence = float(max(model.predict_proba(features)[0]))

return jsonify({
    "prediction": int(prediction),
    "confidence": confidence,
})

if __name__ == "__main__":
    load_model()
    app.run(host="0.0.0.0", port=8080)
```

FILE: tests/__init__.py

FILE: tests/test_preprocess.py

```
"""Unit tests for preprocessing functions."""

import pytest
import pandas as pd
import numpy as np

from src.preprocess import (
    handle_missing_values,
    normalize_features,
    validate_schema,
)

class TestHandleMissingValues:
    def test_median_strategy(self):
        df = pd.DataFrame({"a": [1, 2, np.nan, 4], "b": [10, np.nan, 30, 40]})
        result = handle_missing_values(df, strategy="median")
        assert result["a"].isna().sum() == 0
        assert result["b"].isna().sum() == 0

    def test_mean_strategy(self):
```

```
df = pd.DataFrame({"a": [1, 2, np.nan, 4]})
result = handle_missing_values(df, strategy="mean")
# Mean of [1, 2, 4] = 2.333...
assert abs(result["a"].iloc[2] - 2.333) < 0.01
```

```
def test_drop_strategy(self):
    df = pd.DataFrame({"a": [1, 2, np.nan, 4], "b": [10, 20, 30, 40]})
    result = handle_missing_values(df, strategy="drop")
    assert len(result) == 3
```

```
def test_invalid_strategy_raises(self):
    df = pd.DataFrame({"a": [1, 2, 3]})
    with pytest.raises(ValueError):
        handle_missing_values(df, strategy="invalid")
```

```
class TestNormalizeFeatures:
    def test_normalizes_to_zero_one(self):
        df = pd.DataFrame({"a": [0, 50, 100], "b": [10, 20, 30]})
        result = normalize_features(df, ["a"])
        assert result["a"].min() == 0.0
        assert result["a"].max() == 1.0
```

```
def test_constant_column(self):
    df = pd.DataFrame({"a": [5, 5, 5]})
    result = normalize_features(df, ["a"])
    assert (result["a"] == 0.0).all()
```

```
def test_missing_column_raises(self):
    df = pd.DataFrame({"a": [1, 2, 3]})
    with pytest.raises(KeyError):
        normalize_features(df, ["nonexistent"])
```

```
class TestValidateSchema:
    def test_valid_schema(self):
        df = pd.DataFrame({"a": [1], "b": [2], "c": [3]})
        assert validate_schema(df, ["a", "b", "c"]) is True
```

```
def test_missing_column_raises(self):
    df = pd.DataFrame({"a": [1], "b": [2]})
    with pytest.raises(ValueError, match="Missing columns"):
        validate_schema(df, ["a", "b", "c"])
```

FILE: tests/test_predict.py

```
"""Unit tests for prediction code."""
```

```
import pytest
import numpy as np
```

```
class TestPredictionFormat:
    def test_prediction_is_integer(self):
        """Prediction should be an integer class label."""
        # Simulate model output
        prediction = int(np.random.choice([0, 1]))
        assert isinstance(prediction, int)

    def test_confidence_in_range(self):
        """Confidence should be between 0 and 1."""
        confidence = float(np.random.uniform(0.5, 1.0))
        assert 0.0 <= confidence <= 1.0

    def test_prediction_response_format(self):
        """Prediction response should have required keys."""
        response = {"prediction": 1, "confidence": 0.87}
        assert "prediction" in response
        assert "confidence" in response
        assert isinstance(response["prediction"], int)
        assert isinstance(response["confidence"], float)
```

FILE: scripts/validate_data.py

```
"""Data validation script — runs as part of CI pipeline."""
```

```
import sys
import pandas as pd
```

```
DATA_PATH = "data/sample_data.csv"
```

```
# Define expected schema
EXPECTED_COLUMNS = ["feature_1", "feature_2", "feature_3", "feature_4", "target"]
EXPECTED_TYPES = {
    "feature_1": "float64",
    "feature_2": "float64",
    "feature_3": "float64",
    "feature_4": "float64",
    "target": "int64",
```



```
}  
MAX_NULL_RATE = 0.10 # Max 10% nulls per column  
  
def validate():  
    """Run all data validation checks."""  
    print("=" * 60)  
    print("DATA VALIDATION")  
    print("=" * 60)  
  
    # Load data  
    try:  
        df = pd.read_csv(DATA_PATH)  
        print(f"✓ Data loaded: {len(df)} rows, {len(df.columns)} columns")  
    except FileNotFoundError:  
        print(f"✗ FAIL: Data file not found: {DATA_PATH}")  
        sys.exit(1)  
  
    # Check columns exist  
    missing_cols = set(EXPECTED_COLUMNS) - set(df.columns)  
    if missing_cols:  
        print(f"✗ FAIL: Missing columns: {missing_cols}")  
        sys.exit(1)  
    print(f"✓ Schema valid: All {len(EXPECTED_COLUMNS)} columns present")  
  
    # Check data types  
    for col, expected_type in EXPECTED_TYPES.items():  
        actual_type = str(df[col].dtype)  
        if actual_type != expected_type:  
            print(f"✗ FAIL: Column '{col}' type is {actual_type}, expected {expected_type}")  
            sys.exit(1)  
    print("✓ Data types valid")  
  
    # Check null rates  
    for col in EXPECTED_COLUMNS:  
        null_rate = df[col].isna().mean()  
        if null_rate > MAX_NULL_RATE:  
            print(f"✗ FAIL: Column '{col}' null rate {null_rate:.2%} exceeds {MAX_NULL_RATE:.0%}")  
            sys.exit(1)  
    print(f"✓ Null rates within threshold (<{MAX_NULL_RATE:.0%})")  
  
    # Check minimum row count  
    if len(df) < 100:
```



```
print(f"✗ FAIL: Insufficient data ({len(df)} rows, minimum 100)")
sys.exit(1)
print(f"✓ Sufficient data: {len(df)} rows")

print("=" * 60)
print("ALL DATA VALIDATION CHECKS PASSED")
print("=" * 60)

if __name__ == "__main__":
    validate()
```

FILE: scripts/validate_model.py

```
"""Model validation script — trains and checks performance thresholds."""

import sys
import time

from src.train import train_model

ACCURACY_THRESHOLD = 0.85
DATA_PATH = "data/sample_data.csv"
MODEL_PATH = "model.pkl"

def validate():
    """Train model and validate it meets quality thresholds."""
    print("=" * 60)
    print("MODEL VALIDATION")
    print("=" * 60)

    # Train model
    print("Training model...")
    start = time.time()
    metrics = train_model(DATA_PATH, MODEL_PATH)
    duration = time.time() - start
    print(f"✓ Training complete in {duration:.1f}s")

    # Check accuracy
    accuracy = metrics["accuracy"]
    print(f" Accuracy: {accuracy:.4f} (threshold: {ACCURACY_THRESHOLD})")
    if accuracy < ACCURACY_THRESHOLD:
        print(f"✗ FAIL: Accuracy {accuracy:.4f} below threshold {ACCURACY_THRESHOLD}")
```

```
sys.exit(1)
print(f"✓ Accuracy meets threshold")

# Check model produces predictions
import pickle
import numpy as np

with open(MODEL_PATH, "rb") as f:
    model = pickle.load(f)

test_input = np.random.rand(1, metrics["n_features"])
prediction = model.predict(test_input)
proba = model.predict_proba(test_input)

if prediction is None or len(prediction) == 0:
    print("✗ FAIL: Model produced no prediction")
    sys.exit(1)
print(f"✓ Model produces valid predictions")

if not (0 <= max(proba[0]) <= 1):
    print("✗ FAIL: Prediction probability out of range")
    sys.exit(1)
print(f"✓ Prediction confidence in valid range")

print("=" * 60)
print("ALL MODEL VALIDATION CHECKS PASSED")
print(f" Accuracy: {accuracy:.4f}")
print(f" Features: {metrics['n_features']}")
print(f" Samples: {metrics['n_samples']}")
print("=" * 60)

if __name__ == "__main__":
    validate()
```

FILE: data/sample_data.csv

```
feature_1,feature_2,feature_3,feature_4,target
5.1,3.5,1.4,0.2,0
4.9,3.0,1.4,0.2,0
4.7,3.2,1.3,0.2,0
4.6,3.1,1.5,0.2,0
5.0,3.6,1.4,0.2,0
5.4,3.9,1.7,0.4,0
```



4.6,3.4,1.4,0.3,0
5.0,3.4,1.5,0.2,0
4.4,2.9,1.4,0.2,0
4.9,3.1,1.5,0.1,0
5.4,3.7,1.5,0.2,0
4.8,3.4,1.6,0.2,0
4.8,3.0,1.4,0.1,0
4.3,3.0,1.1,0.1,0
5.8,4.0,1.2,0.2,0
5.7,4.4,1.5,0.4,0
5.4,3.9,1.3,0.4,0
5.1,3.5,1.4,0.3,0
5.7,3.8,1.7,0.3,0
5.1,3.8,1.5,0.3,0
5.4,3.4,1.7,0.2,0
5.1,3.7,1.5,0.4,0
4.6,3.6,1.0,0.2,0
5.1,3.3,1.7,0.5,0
4.8,3.4,1.9,0.2,0
7.0,3.2,4.7,1.4,1
6.4,3.2,4.5,1.5,1
6.9,3.1,4.9,1.5,1
5.5,2.3,4.0,1.3,1
6.5,2.8,4.6,1.5,1
5.7,2.8,4.5,1.3,1
6.3,3.3,4.7,1.6,1
4.9,2.4,3.3,1.0,1
6.6,2.9,4.6,1.3,1
5.2,2.7,3.9,1.4,1
5.0,2.0,3.5,1.0,1
5.9,3.0,4.2,1.5,1
6.0,2.2,4.0,1.0,1
6.1,2.9,4.7,1.4,1
5.6,2.9,3.6,1.3,1
6.7,3.1,4.4,1.4,1
5.6,3.0,4.5,1.5,1
5.8,2.7,4.1,1.0,1
6.2,2.2,4.5,1.5,1
5.6,2.5,3.9,1.1,1
5.9,3.2,4.8,1.8,1
6.1,2.8,4.0,1.3,1
6.3,2.5,4.9,1.5,1



6.1,2.8,4.7,1.2,1
6.4,2.9,4.3,1.3,1
6.6,3.0,4.4,1.4,1
6.8,2.8,4.8,1.4,1
6.7,3.0,5.0,1.7,1
6.0,2.9,4.5,1.5,1
5.7,2.6,3.5,1.0,1
5.5,2.4,3.8,1.1,1
5.5,2.4,3.7,1.0,1
5.8,2.7,3.9,1.2,1
6.0,2.7,5.1,1.6,1
5.4,3.0,4.5,1.5,1
6.0,3.4,4.5,1.6,1
6.7,3.1,4.7,1.5,1
6.3,2.3,4.4,1.3,1
5.6,3.0,4.1,1.3,1
5.5,2.5,4.0,1.3,1
5.5,2.6,4.4,1.2,1
6.1,3.0,4.6,1.4,1
5.8,2.6,4.0,1.2,1
5.0,2.3,3.3,1.0,1
5.6,2.7,4.2,1.3,1
5.7,3.0,4.2,1.2,1
5.7,2.9,4.2,1.3,1
6.2,2.9,4.3,1.3,1
5.1,2.5,3.0,1.1,1
5.7,2.8,4.1,1.3,1
6.3,3.3,6.0,2.5,2
5.8,2.7,5.1,1.9,2
7.1,3.0,5.9,2.1,2
6.3,2.9,5.6,1.8,2
6.5,3.0,5.8,2.2,2
7.6,3.0,6.6,2.1,2
4.9,2.5,4.5,1.7,2
7.3,2.9,6.3,1.8,2
6.7,2.5,5.8,1.8,2
7.2,3.6,6.1,2.5,2
6.5,3.2,5.1,2.0,2
6.4,2.7,5.3,1.9,2
6.8,3.0,5.5,2.1,2
5.7,2.5,5.0,2.0,2
5.8,2.8,5.1,2.4,2



6.4,3.2,5.3,2.3,2
6.5,3.0,5.5,1.8,2
7.7,3.8,6.7,2.2,2
7.7,2.6,6.9,2.3,2
6.0,2.2,5.0,1.5,2
6.9,3.2,5.7,2.3,2
5.6,2.8,4.9,2.0,2
7.7,2.8,6.7,2.0,2
6.3,2.7,4.9,1.8,2
6.7,3.3,5.7,2.1,2
7.2,3.2,6.0,1.8,2
6.2,2.8,4.8,1.8,2
6.1,3.0,4.9,1.8,2
6.4,2.8,5.6,2.1,2
7.2,3.0,5.8,1.6,2
7.4,2.8,6.1,1.9,2
7.9,3.8,6.4,2.0,2
6.4,2.8,5.6,2.2,2
6.3,2.8,5.1,1.5,2
6.1,2.6,5.6,1.4,2
7.7,3.0,6.1,2.3,2
6.3,3.4,5.6,2.4,2
6.4,3.1,5.5,1.8,2
6.0,3.0,4.8,1.8,2
6.9,3.1,5.4,2.1,2
6.7,3.1,5.6,2.4,2
6.9,3.1,5.1,2.3,2
5.8,2.7,5.1,1.9,2
6.8,3.2,5.9,2.3,2
6.7,3.3,5.7,2.5,2
6.7,3.0,5.2,2.3,2
6.3,2.5,5.0,1.9,2
6.5,3.0,5.2,2.0,2
6.2,3.4,5.4,2.3,2
5.9,3.0,5.1,1.8,2

FILE: Dockerfile

```
FROM python:3.11-slim
WORKDIR /app
# Install dependencies
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
```



```
# Copy source code and model
```

```
COPY src/ ./src/
```

```
COPY data/ ./data/
```

```
# Train model during build (for demo — in production, model is pre-trained)
```

```
RUN python -c "from src.train import train_model; train_model('data/sample_data.csv', 'model.pkl')"
```

```
# Copy prediction server
```

```
COPY src/predict.py .
```

```
EXPOSE 8080
```

```
CMD ["python", "predict.py"]
```

SECTION 5: STEP-BY-STEP SETUP

STEP 1: Create GitHub Repository

```
# On GitHub.com: Click "+" → "New repository" → name: "ml-cicd-demo"
```

```
# Initialize with README
```

```
# Clone locally
```

```
git clone https://github.com/YOUR_USERNAME/ml-cicd-demo.git
```

```
cd ml-cicd-demo
```

STEP 2: Create directory structure

```
mkdir -p .github/workflows src tests scripts data
```

STEP 3: Create all files as shown in Section 4

STEP 4: Test locally before pushing

```
# Install dependencies
```

```
pip install -r requirements.txt
```

```
# Run tests
```

```
pytest tests/ -v
```

```
# Run data validation
```

```
python scripts/validate_data.py
```




```
# Run model validation
python scripts/validate_model.py

# Test Docker build
docker build -t ml-model:test .
docker run -d --name test -p 8080:8080 ml-model:test
curl http://localhost:8080/health
docker stop test && docker rm test
```

STEP 5: Push to GitHub (triggers pipeline)

```
git add .
git commit -m "Add ML CI/CD pipeline"
git push origin main
```

STEP 6: Watch pipeline execute

Go to GitHub.com → your repository → "Actions" tab
You should see the workflow running with 3 stages
Each stage shows real-time logs

SECTION 6: VERIFICATION STEPS

6A: Verify Local Tests Pass

```
pytest tests/ -v
```

Expected output:

```
tests/test_preprocess.py::TestHandleMissingValues::test_median_strategy PASSED
tests/test_preprocess.py::TestHandleMissingValues::test_mean_strategy PASSED
tests/test_preprocess.py::TestHandleMissingValues::test_drop_strategy PASSED
tests/test_preprocess.py::TestHandleMissingValues::test_invalid_strategy_raises PASSED
tests/test_preprocess.py::TestNormalizeFeatures::test_normalizes_to_zero_one PASSED
tests/test_preprocess.py::TestNormalizeFeatures::test_constant_column PASSED
tests/test_preprocess.py::TestNormalizeFeatures::test_missing_column_raises PASSED
tests/test_preprocess.py::TestValidateSchema::test_valid_schema PASSED
tests/test_preprocess.py::TestValidateSchema::test_missing_column_raises PASSED
tests/test_predict.py::TestPredictionFormat::test_prediction_is_integer PASSED
tests/test_predict.py::TestPredictionFormat::test_confidence_in_range PASSED
tests/test_predict.py::TestPredictionFormat::test_prediction_response_format PASSED
```

12 passed

6B: Verify Data Validation

python scripts/validate_data.py

Expected output:

DATA VALIDATION

- ✓ Data loaded: 125 rows, 5 columns
- ✓ Schema valid: All 5 columns present
- ✓ Data types valid
- ✓ Null rates within threshold (<10%)
- ✓ Sufficient data: 125 rows

ALL DATA VALIDATION CHECKS PASSED

6C: Verify Model Validation

python scripts/validate_model.py

Expected output:

MODEL VALIDATION

Training model...

- ✓ Training complete in 0.5s
- Accuracy: 0.9600 (threshold: 0.85)
- ✓ Accuracy meets threshold
 - ✓ Model produces valid predictions
 - ✓ Prediction confidence in valid range
- ALL MODEL VALIDATION CHECKS PASSED

Accuracy: 0.9600

Features: 4

Samples: 125

6D: Verify GitHub Actions Pipeline

After pushing:

1. Go to GitHub → repository → "Actions" tab
2. Click on the latest workflow run
3. Verify all 3 jobs show green checkmarks:

test ✓

validate ✓

build ✓



SECTION 7: CLEANUP

Remove local artifacts:

```
rm -f model.pkl  
docker rmi ml-model:test
```

To delete the GitHub repository:

Settings → scroll to bottom → "Delete this repository"

SECTION 8: TROUBLESHOOTING

ISSUE: "Workflow not triggering"

FIX: Ensure file is at exactly .github/workflows/ml-pipeline.yml

Check branch name matches trigger (main vs master)

ISSUE: "ModuleNotFoundError: No module named 'src'"

FIX: Ensure __init__.py exists in src/ and tests/ directories

Run pytest from repository root, not from tests/ folder

For scripts in scripts/ folder, add this at the TOP of each script:

```
import sys, os  
sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
```

This adds the project root to Python's path so "from src.train import ..." works.

ISSUE: "SyntaxError: f-string: unmatched '[' " in Docker build (Python 3.11)

CAUSE: Python 3.12+ allows quotes inside f-string braces, but Python 3.11 does not.

metrics["accuracy"] inside an f-string with double quotes fails on 3.11.

FIX: Use single quotes for dict keys inside f-strings:

WRONG: `print(f"Accuracy: {metrics["accuracy"]}")`

RIGHT: `print(f"Accuracy: {metrics['accuracy']}")`

This works on both Python 3.11 and 3.12+.

ISSUE: "refusing to allow a Personal Access Token to create or update workflow"

CAUSE: GitHub PAT needs the 'workflow' scope to push .github/workflows/ files

FIX: Go to GitHub → Settings → Developer settings → Personal access tokens

Edit your token → check BOTH 'repo' AND 'workflow' scopes → Update token

ISSUE: "Permission denied" when pushing via SSH

CAUSE: SSH key is linked to a different GitHub account than the repo owner

FIX: Use HTTPS instead of SSH:

```
git remote set-url origin https://github.com/USERNAME/ml-cicd-demo.git
```

Then push with username + Personal Access Token (not password)

ISSUE: "pytest: command not found"

FIX: Ensure pytest is in requirements.txt and pip install was run



ISSUE: "Docker build fails in CI"

FIX: Test Dockerfile locally first: docker build -t test .

Check all COPY paths reference files that exist

ISSUE: "Health check fails in CI"

FIX: Ensure model.pkl is created during Docker build

Add sleep before curl to allow Flask server to start

ISSUE: "Permission denied on scripts"

FIX: Ensure Python scripts have proper line endings (LF not CRLF)

Use: git config core.autocrlf input